STUDENT LIBRARY MANAGEMENT SYSTEM

Project Report

A simple terminal-based library management application developed in Python.

Course Project

Submitted as part of the course project requirements

1. Introduction

The Student Library Management System is a small, terminal-based Python application designed to make basic library tasks easier to manage. The project focuses on common operations such as adding books, viewing books, searching for books, borrowing and returning books, and checking the current library status.

The application was intentionally kept simple so that the main concepts remain clear. It uses Python's standard library and stores book information locally in a JSON file. The program is started through a single entry point, **main.py**.

2. Problem Statement

Managing a small collection of books manually can become inconvenient when the number of books increases. It may be difficult to quickly check whether a book is available, find a particular book, or keep track of borrowed books.

This project addresses that problem by providing a simple menu-driven system that keeps basic book information in one place and allows common library operations to be performed from the terminal.

3. Scope and Target Users

Target users: Students and users managing a small personal or academic library.

Scope:

• Add, view and remove books.
• Search books using book ID, title or author.
• Borrow and return books with availability checks.
• View total, available and borrowed book counts.
• Store information locally using JSON.

The project does not include user accounts, a graphical interface, a web application, online services, payment processing, fines, or a database server.

4. Functional Requirements

ID	Requirement
FR-01	Book Management — add, remove and view books.
FR-02	Book Search — search by book ID, title or author.
FR-03	Borrow & Return — change book status after validating availability.
FR-04	Library Status — display total, available and borrowed books.

5. Non-Functional Requirements

• **Usability:** The menu and messages should be easy to understand for a beginner user.
• **Reliability:** Invalid operations should be handled without unexpectedly stopping the program.
• **Performance:** Operations should respond quickly for a small local collection of books.
• **Maintainability:** The code should be readable and organised into clear functions.
• **Portability:** The project should run on systems with a standard Python installation.

6. System Design

The application follows a simple flow. When the program starts, it loads the existing book data from the JSON file. The user is then shown a menu and selects an operation. The selected operation validates its input, updates the data when necessary, saves the changes, and returns to the menu.

Basic workflow:
Start → Load Books → Display Menu → Select Operation → Validate Input → Perform Operation → Save Data → Display Result → Return to Menu → Exit

The system is deliberately kept as a single Python application with supporting data and documentation files. This matches the project's beginner-level scope while still separating the working program from its supporting artifacts.

7. Data Design

Book records are stored in **data/books.json**. Each book contains four basic fields:

Field	Purpose
book_id	Unique identifier for the book.
title	Title of the book.
author	Author name.
status	Current state: Available or Borrowed.

8. Implementation

The complete application is contained in **main.py**. The program uses the Python standard library, mainly the **json** and **os** modules.

The main functions are:

• **load_books()** — loads book records from the JSON file.
• **save_books()** — saves the current book list.
• **add_book()** — adds a new book after validation.
• **view_books()** — displays all books.
• **search_books()** — searches by ID, title or author.
• **borrow_book()** / **return_book()** — manage book status.
• **remove_book()** — removes a selected book.
• **show_library_status()** — displays library counts.

Input validation is included for empty values, duplicate book IDs, invalid menu choices, missing book IDs, and invalid borrow or return operations.

9. Testing Approach

Testing was planned around the main functions of the application rather than only checking whether the program starts. Each major operation can be tested with both normal and invalid inputs.

Test	Expected result
Add a new book	Book is added and stored.
Add duplicate ID	User receives an error; duplicate is not added.
Search existing book	Matching book information is displayed.
Borrow available book	Status changes to Borrowed.
Borrow borrowed book	Operation is rejected.
Return borrowed book	Status changes to Available.
Return available book	Operation is rejected.
Remove existing book	Book is removed.

Invalid menu choice	User is asked to enter a valid option.

10. Results

The completed system provides the planned core library operations through a terminal menu. Book data can be retained between program runs through the JSON file, so the user does not have to re-enter the complete book list every time.

The project remains small enough to understand and demonstrate in a classroom setting while covering input handling, functions, lists and dictionaries, file handling, JSON data, conditions, loops, validation, and basic error handling.

11. Challenges and Learnings

One of the main challenges was keeping the program simple while still handling incorrect user input and different book states correctly. The project also required thinking about how data should be stored so that changes remain available after the program closes.

The project provided practical experience with designing a small problem-solving application, breaking a task into functions, validating inputs, reading and writing files, and testing different user scenarios.

12. Future Enhancements

Possible future improvements include student/member records, borrowing history, due dates, fine calculation, better reporting, and a graphical interface. These features are outside the current project's intended scope but could be added in a larger version.

13. Conclusion

The Student Library Management System demonstrates how basic Python concepts can be combined to solve a practical problem. The final application is intentionally straightforward, with a single executable Python file, local JSON storage, input validation, and clear library operations.

14. References

• Project requirements document provided for the course project.
• Python Standard Library documentation for JSON and file handling.

Project structure and supporting design artifacts are included with the project submission.